EXPERIMENT 9

Explaining Movie-Review Sentiment Predictions Using LIME

Student Detail	Information
Name	Raj Mathuria
UID	2023300139
Batch	PE - C
GitHub Repository	https://github.com/CodeCraftsmanRaj/xAI/tree/main/experiment_9_lime_complete

Aim

To apply Local Interpretable Model-agnostic Explanations (LIME) to identify which words in a movie-review sentence support or oppose a sentiment model's positive prediction, and to compare LIME's local word scores with direct model predictions after removing one word at a time.

Objectives

- Load the pretrained DistilBERT sentiment classifier and identify its output-label mapping.
- Obtain class probabilities for a review sentence.
- Use LIME to estimate the local contribution of individual words.
- Inspect randomly perturbed sentences and the three lowest- and highest-positive-probability perturbations.
- Compare LIME scores with one-word-removal tests.
- Repeat the explanation with 100 samples and compare it with the 1,000-sample explanation.

Software and Model

Model: distilbert/distilbert-base-uncased-finetuned-sst-2-english

Implementation: Python, PyTorch, Hugging Face Transformers, LIME, pandas, NumPy, and PyYAML.

The model was executed on CUDA according to the run log. The detected label mapping was 0 = NEGATIVE and 1 = POSITIVE.

Theory

LIME explains an individual prediction by creating perturbed versions of the input, obtaining predictions for those versions, and fitting a simple interpretable model around the input of interest. For text, words are removed or masked to create local variations. The resulting signed weights approximate how each word affects the selected class locally. A positive LIME weight supports the class being explained (POSITIVE in this experiment); a negative weight pushes away from that class. These weights are not class probabilities and should not be interpreted as causal effects.

Procedure

1. Load the configured pretrained tokenizer and sequence-classification model.
2. Read the model's id2label mapping and create a prediction function that returns probabilities for both classes.
3. Predict the required sentence: "The plot was weak, but the acting was wonderful."

4. Run LIME with 1,000 samples and record the word weights.
5. Save the original sentence, eight randomly selected perturbed copies, and the three perturbations with the lowest and highest positive-class probabilities.
6. Remove one whitespace-delimited word at a time and record the resulting class probabilities.
7. Repeat the LIME explanation with 100 samples.
8. Repeat the analysis for a custom sentence containing repeated negation.
9. Save all tables, model-call records, summary JSON, HTML explanations, and logs in the results directory.

Results and Observations

Sentence 1

The plot was weak, but the acting was wonderful.

Class	Predicted probability
NEGATIVE	0.00016819 (0.0168%)
POSITIVE	0.99983180 (99.9832%)

Explained class: POSITIVE. LIME sample counts compared: 1000 and 100.

Word	LIME weight (1,000 samples)	Interpretation
wonderful	+0.76925	Supports POSITIVE
weak	-0.09878	Opposes POSITIVE
plot	-0.07136	Opposes POSITIVE
acting	+0.06936	Supports POSITIVE
was	-0.04427	Opposes POSITIVE
but	-0.02467	Opposes POSITIVE
the	+0.01662	Supports POSITIVE
The	+0.01252	Supports POSITIVE

Largest positive weight: “wonderful” (+0.76925). Largest negative weight: “weak” (-0.09878).

Rank consistency between 1,000 and 100 samples (Spearman correlation over shared words): 0.857.

Sentence 2

The movie was not good, not good at all, despite the beautiful soundtrack.

Class	Predicted probability
NEGATIVE	0.99941015 (99.9410%)
POSITIVE	0.00058988 (0.0590%)

Explained class: POSITIVE. LIME sample counts compared: 1000 and 100.

Word	LIME weight (1,000 samples)	Interpretation
not	-0.74924	Opposes POSITIVE
good	+0.14536	Supports POSITIVE
despite	-0.07306	Opposes POSITIVE
beautiful	+0.05639	Supports POSITIVE
soundtrack	+0.02866	Supports POSITIVE
was	-0.02510	Opposes POSITIVE
the	-0.01655	Opposes POSITIVE
The	-0.01578	Opposes POSITIVE
at	-0.01100	Opposes POSITIVE
movie	+0.01013	Supports POSITIVE
all	+0.00117	Supports POSITIVE

Largest positive weight: “good” (+0.14536). Largest negative weight: “not” (-0.74924).

Rank consistency between 1,000 and 100 samples (Spearman correlation over shared words): 0.573.

One-Word-Removal Analysis

The table below summarizes the most informative one-word removals. Removing a word changes the input context, so the probability difference is a direct model response rather than a LIME weight.

Sentence	Removed word	P(POSITIVE) after removal	Change in P(POSITIVE)
Required sentence	The	0.999820	-0.000012
Required sentence	plot	0.999845	+0.000013
Required sentence	was	0.999842	+0.000010
Required sentence	weak,	0.999829	-0.000003
Required sentence	but	0.999867	+0.000035
Required sentence	the	0.999803	-0.000029
Required sentence	acting	0.999837	+0.000005
Required sentence	was	0.999801	-0.000031
Required sentence	wonderful.	0.010959	-0.988873
Custom sentence	The	0.000624	+0.000035
Custom sentence	movie	0.001224	+0.000634
Custom sentence	was	0.000597	+0.000008
Custom sentence	not	0.999016	+0.998427
Custom sentence	good,	0.000716	+0.000126
Custom sentence	not	0.001455	+0.000865
Custom sentence	good	0.000927	+0.000337
Custom sentence	at	0.000718	+0.000128
Custom sentence	all,	0.000629	+0.000039
Custom sentence	despite	0.000245	-0.000345
Custom sentence	the	0.000756	+0.000166
Custom sentence	beautiful	0.000241	-0.000349
Custom sentence	soundtrack.	0.000343	-0.000246

Answers to the Task Questions

1. Which word has the largest positive score and which has the largest negative?

For the required sentence, “wonderful” had the largest positive weight (+0.76925), while “weak” had the largest negative weight (−0.09878). For the custom sentence, “good” had the largest positive weight (+0.14536) and “not” had the largest negative weight (−0.74924). Because the custom explanation targets POSITIVE, the strong negative weight for “not” is consistent with the model interpreting the repeated negation as negative sentiment.

2. Do the removed words in the lowest-probability copies match LIME’s top words?

Partly, but not uniformly. In the required sentence, the lowest-positive-probability copies remove “wonderful,” which agrees with LIME identifying it as the strongest positive word. In the custom sentence, the lowest-probability copies retain the repeated “not good” phrase and remove other words such as “beautiful” and “soundtrack.” This is consistent with the negative phrase remaining influential, but it does not mean the removed words directly match LIME’s strongest negative word (“not”). Perturbations remove combinations of words, so this comparison is qualitative.

3. Does the remove-one-word table rank words in the same order as LIME?

Not exactly, but the main signals agree. In the required sentence, removing “wonderful” drops P(POSITIVE) from about 0.999832 to 0.010959, strongly supporting LIME’s identification of “wonderful” as the dominant positive word. Removing “weak” barely changes the already saturated probability, so the direct-removal ranking does not reproduce

all LIME weights. In the custom sentence, removing the first “not” changes P(POSITIVE) from about 0.000590 to 0.999016, showing that negation is critical. However, removing one occurrence of a repeated word is not equivalent to removing every occurrence, and the remaining sentence still changes context.

4. Do scores change when samples are reduced from 1,000 to 100?

Yes. For the required sentence, “wonderful” remained the strongest positive word (0.76925 at 1,000 samples versus 0.75369 at 100), and the shared-word Spearman rank correlation was 0.857, indicating fairly stable rankings. For the custom sentence, “not” remained the strongest negative word (−0.74924 versus −0.54872), but the rank correlation fell to 0.573, indicating greater sensitivity to the smaller sample count. Thus, the main interpretation was stable, but some secondary word scores and ranks changed, especially for the custom sentence.

Discussion and Limitations

- The required sentence was predicted as POSITIVE with probability 0.999832, while the negation-heavy custom sentence was predicted as NEGATIVE with probability 0.999410.
- LIME successfully highlighted semantically meaningful terms: “wonderful” supports positive sentiment, “weak” opposes it, and “not” strongly opposes a positive prediction in the custom sentence.
- The model’s probabilities are highly saturated for the original sentences. Consequently, removing many words has tiny probability effects unless a highly influential word is removed.
- LIME perturbations are generated by removing words and can contain multiple spaces or incomplete fragments. These are artificial probes rather than grammatical reviews.
- The custom sentence’s 100-sample ranking is less stable than its 1,000-sample ranking. More samples generally give a more reliable local approximation, at greater computational cost.
- The one-word-removal experiment uses whitespace-delimited words, so punctuation remains attached to tokens such as “weak,” and “wonderful.” The repeated word “not” is tested at each position separately.

Conclusion

LIME was applied successfully to the DistilBERT sentiment classifier to explain two movie-review sentences. For the required sentence, “wonderful” was the strongest positive contributor and “weak” the strongest negative contributor. For the custom sentence, “not” was the strongest word opposing a positive prediction, and removing a single occurrence of “not” caused the model’s prediction to shift sharply toward POSITIVE. The 100-sample explanation preserved the main words but changed some secondary weights and rankings, especially for the negation-heavy example. Overall, LIME provided useful local explanations, while the direct-removal test demonstrated that context and word interactions can make the two methods differ.

Generated Result Files

The submitted results archive contains summary.json, model_calls.csv, LIME score CSV files for both sample counts and both sentences, selected perturbation CSV files, one-word-removal CSV files, HTML explanations, and run.log. The full perturbation and per-word tables are retained in these files for reproducibility.

GitHub Repository: https://github.com/CodeCraftsmanRaj/xAI/tree/main/experiment_9_lime_complete